These findings match intuition: we might expect that on the easy questions, the verifier will make mostly correct assessments of correctness. Therefore, by applying further optimization via beam search, we only further amplify any spurious features learned by the verifier, causing performance degredation. On the more difficult questions, the base model is much less likely to sample the correct answer in the first place, so search can serve to help guide the model towards producing the correct answer more often.

Compute-optimal search. Given the above results, it is clear that question difficulty can be a useful statistic to predict the optimal search strategy to use at a given compute budget. Additionally, the best choice of search strategy can vary drastically as a function of this difficulty statistic. We therefore visualize the "compute-optimal" scaling trend, as represented by the best performing search strategy at each difficulty level in Figure 4. We see that in the low generation budget regime, using both the oracle and predicted difficulty, **compute-optimal scaling can nearly outperform best-of-N using up to *4x* less test-time compute** (e.g. 16 verses 64 generations). While in the higher budget regime, some of these benefits diminish with the use of predicted difficulty, with oracle bins we still see continued improvements from optimally scaling test-time compute. This result demonstrates the performance gains that could be obtained by adaptively allocating test-time compute during search.

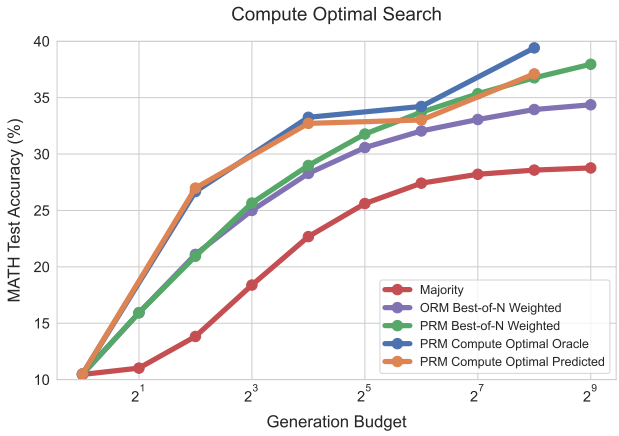

Figure 4 | ***Comparing compute-optimal test-time compute allocation against baselines with PRM search.*** By scaling test time compute per the notion of question difficulty, we find that we can nearly outperform PRM best-of-N using up to **4**× less test-time compute (e.g. 16 verses 64 generations). "**Compute-optimal oracle**" refers to using oracle difficulty bins derived from the groundtruth correctness information, and "**compute-optimal predicted**" refers to using the PRM's predictions to generate difficulty bins. Observe that the curves with either type of difficulty bins largely overlap with each other.

Takeaways for compute-optimal scaling of verifiers

We find that the efficacy of any given verifier search method depends critically on both the compute budget and the question at hand. Specifically, beam-search is more effective on harder questions and at lower compute budgets, whereas best-of-N is more effective on easier questions and at higher budgets. Moreover, by selecting the best search setting for a given question difficulty and test-time compute budget, we can nearly outperform best-of-N using up to *4x* less test-time compute.

6. Refining the Proposal Distribution

So far, we studied the test-time compute scaling properties of search against verifiers. Now we turn to studying the scaling properties of modifying the proposal distribution (Section 2). Concretely, we enable the model to revise their own answers iteratively, allowing the model to dynamically improve it's own distribution at test time. Simply prompting existing LLMs to correct their own mistakes tends to be largely ineffective for obtaining performance improvements on reasoning problems [15]. Therefore, we build on the recipe prescribed by Qu et al. [28], incorporate modifications for our setting, and finetune language models to iteratively revise their own answers. We first describe how we train and use models that refine their own proposal distribution by sequentially conditioning on their own previous attempts at the question. We then analyze the inference-time scaling properties of revision models.

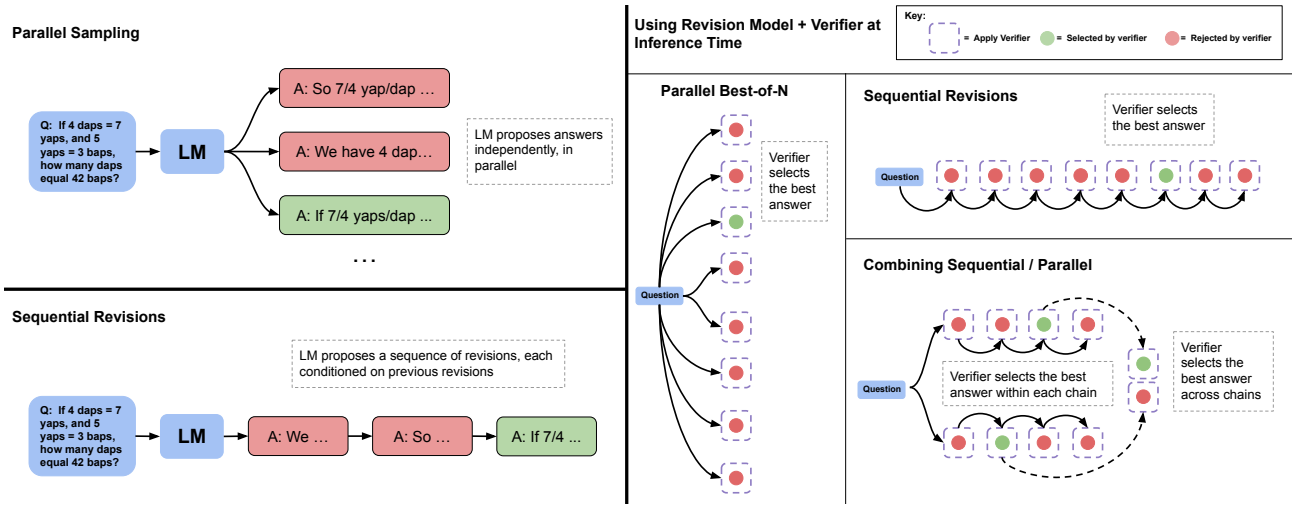

Figure 5 | ***Parallel sampling (e.g., Best-of-N) verses sequential revisions.*** **Left:** Parallel sampling generates N answers independently in parallel, whereas sequential revisions generates each one in sequence conditioned on previous attempts. **Right:** In both the sequential and parallel cases, we can use the verifier to determine the best-of-N answers (e.g. by applying best-of-N weighted). We can also allocate some of our budget to parallel and some to sequential, effectively enabling a combination of the two sampling strategies. In this case, we use the verifier to first select the best answer within each sequential chain and then select the best answer accross chains.

6.1. Setup: Training and Using Revision Models

Our procedure for finetuning revision models is similar to [28], though we introduce some crucial differences. For finetuning, we need trajectories consisting of a sequence of incorrect answers followed by a correct answer, that we can then run SFT on. Ideally, we want the correct answer to be *correlated* with the incorrect answers provided in context, so as to effectively teach the model to *implicitly* identify mistakes in examples provided in-context, followed by correcting those mistakes by making edits as opposed to ignoring the in-context examples altogether, and trying again from scratch.

Generating revision data. The on-policy approach of Qu et al. [28] for obtaining several multi-turn rollouts was shown to be effective, but it was not entirely feasible in our infrastructure due to compute costs associated with running multi-turn rollouts. Therefore, we sampled 64 responses *in parallel* at a higher temperature and post-hoc constructed multi-turn rollouts from these independent samples. Specifically, following the recipe of [1], we pair up each correct answer with a sequence of incorrect answers from this set as context to construct multi-turn finetuning data. We include up to four incorrect answers in context, where the specific number of solutions in context is sampled randomly from a uniform distribution over categories 0 to 4. We use a character edit distance metric to prioritize selecting incorrect answers which are correlated with the final correct answer (see Appendix H). Note that token edit distance is not a perfect measure of correlation, but we found this heuristic to be sufficient to correlate incorrect in-context answers with correct target answers to facilitate training a meaningful revision model, as opposed to randomly pairing incorrect and correct responses with uncorrelated responses.

Using revisions at inference-time. Given a finetuned revision model, we can then sample a sequence of revisions from the model at test-time. While our revision model is only trained with up to four previous answers in-context, we can sample longer chains by truncating the context to the most recent four revised responses. In Figure 6 (left), we see that as we sample longer chains from the revision model, the model's pass@1 at each step gradually improves, demonstrating that we are able to effectively teach the model to learn from mistakes made by previous answers in context.

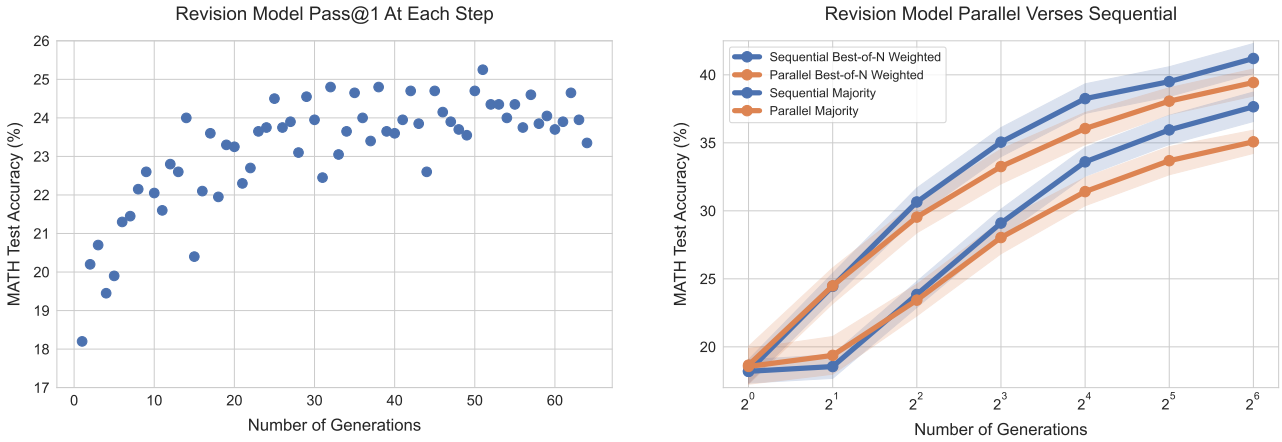

Figure 6 | **Left:** ***Our revision model's pass@1 at each revision step.*** Pass@1 gradually improves after each revision step, even improving beyond the 4 revision steps that it was trained for. We estimate pass@1 at each step by by averaging over the performance of 4 revision trajectories of length 64 for each question in the test-set. **Right:** ***Sequential vs parallel sampling from the revision model.*** Comparing performance when generating N initial answers in parallel from our revision model, verses generating N revisions sequentially, with the model. When using both the verifier and majority voting to select the answer, we see that generating answers sequentially with the revision model narrowly outperforms generating them in parallel.

That said, there is a distribution shift at inference time: the model was trained on only sequences with incorrect answers in context, but at test-time the model may sample correct answers that are included in the context. In this case, it may incidentally turn the correct answer into an incorrect answer in the next revision step. We find that indeed, similar to Qu et al. [28], around 38% of correct answers get converted back to incorrect ones with our revision model using a naïve approach. Therefore, we employ a mechanism based on sequential majority voting or verifier-based selection to select the most correct answer from the sequence of revisions made by the model (see Figure 5) to produce the best answer.

Comparisons. To test the efficacy of modifying the proposal distribution via revisions, we setup an even comparison between the performance of sampling N revisions in sequence and sampling N attempts at a question in parallel. We see in Figure 6 (right), that with both the verifier-based and majority-based selection mechanisms sampling solutions in sequence outperforms sampling them in parallel.

6.2. Analysis Results: Test-Time Scaling with Revisions

We saw previously that proposing answers sequentially outperforms proposing them in parallel. However, we might expect sequential and parallel sampling to have different properties. Sampling answers in parallel may act as more of a global search process, that could in principle, provide coverage over many totally different approaches for solving a problem, for instance, different candidates might utilize different high-level approaches altogether. Sequential sampling, on the other hand, may work more as a local refinement process, revising responses that are already somewhat on the right track. Due to these complementary benefits, we should strike a balance between these two extremes by allocating some of our inference-time budget to parallel sampling (e.g. $\sqrt{N}$) and the rest to sequential revisions (e.g. $\sqrt{N}$). We will now show the existence of a compute-optimal ratio between sequential and parallel sampling, and understand their relative pros and cons based on difficulty of a given prompt.

Trading off sequential and parallel test-time compute. To understand how to optimally allocate sequential and parallel compute, we perform a sweep over a number of different ratios. We see, in Figure 7 (left), that indeed, at a given generation budget, **there exists an ideal sequential to parallel ratio, that**